# Non-Negative Conjugate Gradients

Thomas Schmelzer[1] and Martin Stoll[2]

[1]Jebel Quant Research, Abu Dhabi
[2]Faculty of Mathematics, TU Chemnitz

July 19, 2026

**Abstract**

The conjugate gradient method (CG) solves a symmetric positive definite (SPD) linear system $Ax = b$. On its own it does not respect the inequality $x \geq 0$. This note develops *non-negative conjugate gradients*, a solver for the bound-constrained quadratic

$$\min_x \ \tfrac{1}{2}\, x^\top A x - b^\top x \quad \text{subject to} \quad x \geq 0, \ \ Bx = c, \qquad A \succ 0.$$

It enforces $x \geq 0$ by wrapping CG in an outer active-set loop, and it retains CG's $O(\sqrt{\kappa})$ Krylov convergence, where $\kappa = \kappa(A)$ is the spectral condition number. The equality system $Bx = c$ is present in the equality-augmented variant and absent in the plain bound-constrained and least-squares forms.

The wrapper is a primal-dual active-set method. It solves the unconstrained system over a working set of *free* variables, releases any that come back negative (the primal step), re-admits any bound variable whose reduced gradient is negative (the dual step), and restarts CG on the modified free set each time. These toggles are exactly the principal pivots of the linear complementarity problem $(A, -b)$. Because $A$ is a $P$-matrix, every principal submatrix is invertible and each pivot is well defined.

Our central result concerns termination. We guard a fast block-pivot path with a least-index Bland fallback, and we prove that the loop reaches the unique global minimiser in finitely many outer steps. The proof needs *no* non-degeneracy assumption, the hypothesis that classical block-principal-pivoting termination arguments require. A synthetic study shows that the fallback is never entered on generic data, yet it is genuinely necessary. On an adversarial family of anti-correlated designs the unguarded batch path cycles, and the fallback is what terminates it.

The guarantee is robust to inexactness. Once CG's residual tolerance is tied to the decision threshold, the inexact loop provably visits the same free sets as the exact one. The guarantee attaches to the outer loop rather than the inner solver, so the active-set method stands as an intermediate result in its own right. The same loop, with the same termination proof, accepts a direct factorisation of each free-set system in place of CG. It therefore combines just as readily with direct methods when the reduced blocks are small or structured. We analyse a loop-free projected-gradient reference method alongside it. That method enforces the constraints simultaneously, but it forgoes the Krylov subspace and so converges at the slower $O(\kappa)$ rate.

Each inner solve is matrix-free. When $A = M^\top M$, the operator $v \mapsto M^\top(Mv)$ is applied without ever forming $A$, at two products with $M$ per iteration and no $O(n^2)$ storage. Any regularisation or preconditioner that compresses the spectrum lowers the iteration count. This includes a ridge/Tikhonov split $A \mapsto (1-\alpha)A + \alpha R^\top R$ that also secures the $P$-matrix property. On synthetic SPD problems the loop reaches the planted optimum in at most 6 outer steps. With a direct free-set solve it outpaces Lawson–Hanson and an interior-point solver by more than an order of magnitude on dense instances.

# 1 Introduction

The conjugate gradient method (CG) of Hestenes and Stiefel [16] is the canonical Krylov solver for a symmetric positive definite (SPD) linear system $Ax = b$; see, e.g., Golub and Van Loan [10], Greenbaum [12], or Trefethen and Bau [29]. It builds its iterates in the Krylov subspace $\mathcal{K}_k(A, b)$ and converges at the $O(\sqrt{\kappa})$ rate, where $\kappa = \kappa(A)$ is the spectral condition number (Proposition 3.1). CG solves linear systems; it is *not* designed for inequalities, that is, for constrained programs. This note applies CG to the bound-constrained quadratic

$$\min_{x \in \mathbb{R}^n} \; f(x) = \tfrac{1}{2}\, x^\top A x - b^\top x \quad \text{subject to} \quad x \geq 0, \qquad A \succ 0, \tag{1}$$

without giving up its per-iteration cost or its Krylov convergence rate.

Problem (1) is the strictly convex non-negative quadratic program. When $A = M^\top M$ is the Gram matrix of $M \in \mathbb{R}^{m \times n}$ and $b = M^\top d$, its minimiser solves the non-negative least-squares (NNLS) problem $\min_{x \geq 0} \|Mx - d\|_2^2$, the setting of Lawson and Hanson [20]. Without the constraint, the minimiser is simply $x = A^{-1}b$, which is exactly what CG computes; the constraint $x \geq 0$ is what stands between that clean solve and the answer. We also treat the *equality-augmented* variant, in which a linear equality system $Bx = c$ (with $B \in \mathbb{R}^{p \times n}$ of full row rank) is imposed alongside $x \geq 0$. This variant arises whenever the solution must satisfy a family of linear balance conditions, the single normalisation $\mathbf{1}^\top x = \beta$ being the case $p = 1$. Its multipliers $\lambda \in \mathbb{R}^p$ can be eliminated analytically through a $p \times p$ Schur complement (cf. Section 3).

This note studies the constraint-handling layer directly, and its central contribution is a *termination guarantee*. The vehicle is a *primal-dual active-set* outer loop [24] that wraps CG. It solves the unconstrained system over a working set of *free* variables, releases any variable that returns negative (the primal step), re-admits any bound variable whose reduced gradient is negative (the dual step), and restarts CG on the modified free set each time. The variable toggles are precisely the *principal pivots* of the linear complementarity problem (LCP) $(A, -b)$. Since $A \succ 0$ makes it a $P$-matrix, every principal submatrix is invertible and each pivot is well defined [22, 18, 19]. Guarding a fast block-pivot path with a least-index Bland-style fallback then yields *unconditional* finite termination at the unique global minimiser, with no non-degeneracy hypothesis (Theorem 4.1). Classical block-principal-pivoting termination arguments rely on exactly that hypothesis. A perturbation lemma extends the guarantee to the inexact CG solves the loop actually performs: once the residual is small against the problem's decision margin, the inexact loop visits exactly the free sets of the exact one (Lemma 4.1). Combining this guarantee with a Krylov inner solver on a changing free set turns it into a fast solver in practice. In concrete terms, CG restarts on each reduced system and warm-starts across a parametric family of problems.

The inner solve preserves two properties that matter for speed. First, it is *matrix-free*: when $A = M^\top M$, the action $v \mapsto A_{\mathcal{F}} v = M_{\mathcal{F}}^\top (M_{\mathcal{F}} v)$ on the free set $\mathcal{F}$ is evaluated as two products with $M_{\mathcal{F}}$, so $A$ is *never* assembled and no $O(n^2)$ storage is incurred. Second, its iteration count is governed by the condition number $\kappa$, so any preconditioner that compresses the spectrum accelerates convergence. A ridge/Tikhonov splitting $A \mapsto A_\alpha = (1 - \alpha)A + \alpha R^\top R$ with $R^\top R \succ 0$ does both. It lowers $\kappa$ (Proposition 5.1), and, by keeping every principal submatrix strictly positive definite, it secures the $P$-matrix property that the finite-termination proof relies on (Section 5).

For contrast we analyse a loop-free reference method that enforces the constraint at every step. This *projected-gradient* method takes a gradient step and projects onto the feasible set. The feasible set is the non-negative orthant for (1), or the simplex slice $\Delta_\beta = \{x : \mathbf{1}^\top x = \beta,\, x \geq 0\}$ for the single-normalisation case, projected onto in $O(n \log n)$ by a sort-and-threshold pass (cf. [9, 6]). The method is single-stage and simple. It is not a Krylov subspace method, however: it does not

orthogonalise residuals against prior search directions, and it converges at the $O(\kappa)$ rate, a factor $\sqrt{\kappa}$ slower than the CG inner loop (Section 4).

Both building blocks are individually classical. One is an active-set method and a block-principal-pivoting method for the non-negative and complementarity problems [20, 18, 19, 22, 24]. The other is CG with preconditioning for SPD systems [16, 10, 12, 29, 2]. What is *not* classical is the termination result. Block principal pivoting is classically shown to terminate only under a non-degeneracy assumption, and the least-index Bland fallback removes it (Theorem 4.1). The synthetic study of Section 6 then shows this unconditional guarantee to be dormant on generic data yet genuinely necessary: on an adversarial family of anti-correlated designs the unguarded batch path cycles, and only the fallback terminates it. Wrapping this guaranteed loop around a matrix-free Krylov inner solver, warm-started across a parametric sweep, is the engineering that makes the guaranteed method a fast one as well. We develop and analyse it here on its own, independent of any application.[1]

Driving CG through a changing set of free variables has a lineage of its own, and the distinction from it is worth drawing precisely. Bertsekas' projected Newton method [3], Moré and Toraldo's GPCG [21], and Dostál's proportioning and MPRGP algorithms [7, 8] are *feasible-point* methods. Every iterate satisfies the bounds, and the working face changes through gradient projections. The switches between projection and CG phases are governed by sufficient-decrease or proportioning tests. Their convergence rates are stated in terms of $\kappa$, and they identify the optimal face finitely under a non-degeneracy (strict-complementarity) assumption. The loop developed here is instead a *complementary-basis* method. Its iterates are infeasible between outer steps, and the free set changes by principal pivots driven by signs alone, with no line search or decrease test. Termination is combinatorial: it rests on Murty's least-index rule, which is what removes the non-degeneracy hypothesis. On the NNLS side, Bro and De Jong's fast NNLS [4] accelerates Lawson–Hanson by caching normal-equations cross-products, but it still assembles $M^\top M$ and solves each working-set system by dense factorisation. The present method forms neither, reaching $A$ only through the two operator applications of Section 3.

Section 2 states the problem and its KKT/complementarity structure. Section 3 reduces it to a sequence of unconstrained SPD solves, records the matrix-free CG operator and its convergence rate, and eliminates the equality multipliers analytically. Section 4 develops the active-set loop and the projected-gradient reference method and proves finite termination. Section 5 treats regularisation and preconditioning. Section 6 records the complexity of each method and discusses its behaviour on synthetic SPD test problems.

## 2 Problem and Complementarity Structure

We consider the strictly convex non-negative quadratic program (1),

$$\min_{x\geq 0}\ f(x) = \tfrac{1}{2}\, x^\top A x - b^\top x, \qquad A = A^\top \succ 0, \quad b \in \mathbb{R}^n,$$

together with its two structural specialisations. In the *least-squares* form $A = M^\top M$ and $b = M^\top d$ for $M \in \mathbb{R}^{m\times n}$ of full column rank, so that $f(x) = \frac{1}{2}\|Mx - d\|_2^2 - \frac{1}{2}\|d\|_2^2$ and (1) is NNLS [20]. In the *equality-augmented* form a linear equality system $Bx = c$ is imposed as well,

$$\min_{x}\ \tfrac{1}{2}\, x^\top A x - b^\top x \quad \text{subject to} \quad Bx = c,\ x \geq 0, \qquad B \in \mathbb{R}^{p\times n},\ c \in \mathbb{R}^p, \tag{2}$$

[1] A reference implementation is released as the open-source package `nncg`: https://github.com/Jebel-Quant/nncg. Its continuous-integration test suite is the synthetic study of Section 6.

with $B$ of full row rank $p \le n$. This constrains $x$ to the polytope $\mathcal{P} = \{x \ge 0 : Bx = c\}$, an affine slice of the non-negative orthant. The single normalisation $\mathbf{1}^\top x = \beta$ is the case $p = 1$, $B = \mathbf{1}^\top$, $c = \beta$. A general $B$ carries any finite family of linear balance conditions (group budgets, factor neutralities, netting rules) at the same cost structure.

**Optimality and complementarity.** Because $f$ is strictly convex and the feasible set of (1) is a non-empty closed convex cone, a unique minimiser exists and the Karush-Kuhn-Tucker (KKT) conditions are necessary and sufficient. Introduce a multiplier $s \ge 0$ for $x \ge 0$. Stationarity of the Lagrangian $\frac{1}{2}x^\top Ax - b^\top x - s^\top x$ gives the reduced gradient $s = \nabla f(x) = Ax - b$, and the KKT system is the linear complementarity problem LCP($A$, $-b$):

$$s = Ax - b, \qquad x \ge 0, \qquad s \ge 0, \qquad x_i\, s_i = 0 \ \ \forall i. \tag{3}$$

Complementary slackness partitions the indices into a *free* set $\mathcal{F} = \{i : x_i > 0\}$, on which $s_i = 0$ and hence $(Ax)_i = b_i$, and a *bound* set $\mathcal{B} = \{i : x_i = 0\}$, on which $s_i = (Ax)_i - b_i \ge 0$. A bound variable with $s_i < 0$ would lower $f$ if released to a small positive value; a free variable with $x_i < 0$ is infeasible and must be pushed to the bound. These two violations drive the primal and dual steps of Section 4.

For the equality-augmented problem (2) the Lagrangian carries an additional term $-\lambda^\top(Bx - c)$ with a multiplier $\lambda \in \mathbb{R}^p$. The reduced gradient becomes $s = Ax - b - B^\top\lambda$, and the free-set stationarity reads $(Ax)_i = b_i + (B^\top\lambda)_i$ for $i \in \mathcal{F}$. On the free set the gradient is thus pinned to the row space of $B$, with $\lambda$ the vector of shadow prices of the $p$ balance conditions. The KKT system is now a *mixed* complementarity problem: the free block carries the equality $B_{\mathcal{F}} x_{\mathcal{F}} = c$ alongside the sign conditions. Section 3 shows that $\lambda$ need not be carried as an unknown in the linear solve; it is recovered in closed form from $p + 1$ SPD solves and a $p \times p$ Schur system (for $p = 1$, the two solves of the single-normalisation case). This requires $B_{\mathcal{F}}$ to have full row rank. That holds whenever the free set is large enough, $|\mathcal{F}| \ge p$, and generically thereafter; for $p = 1$ it holds for every non-empty free set.

**The $P$-matrix property.** Since $A \succ 0$, every principal submatrix $A_{\mathcal{F}} = A_{\mathcal{F},\mathcal{F}}$ is itself SPD and hence invertible with $\det A_{\mathcal{F}} > 0$; a matrix all of whose principal minors are positive is a $P$-matrix [22]. Two consequences follow, both used later. First, the linear system $A_{\mathcal{F}}\, x_{\mathcal{F}} = b_{\mathcal{F}}$ solved on any candidate free set is well posed, so CG applies at every outer step (Section 3). Second, LCP($A$, $-b$) with a $P$-matrix has a unique solution reachable by principal pivoting [22, 18], which is the backbone of the finite-termination guarantee in Section 4. When $A$ is only positive semidefinite (as in a rank-deficient least-squares problem, $m < n$), the regularisation of Section 5 restores $A \succ 0$ and with it the $P$-matrix property.

The complementarity structure (3) reduces the search for $x$ to a search over free sets $\mathcal{F}$. Once $\mathcal{F}$ is fixed, the sign constraints drop out, and what remains on $\mathcal{F}$ is exactly the linear system (4) shown well posed above. Section 3 makes this reduction precise, together with the closed-form elimination of $\lambda$ for the equality-augmented problem (2).

## 3 Reduction to Unconstrained SPD Solves

We now fix a candidate free set $\mathcal{F} \subseteq \{1, \dots, n\}$ and set $x_i = 0$ for $i \notin \mathcal{F}$. On $\mathcal{F}$ the bound-constrained problem (1) reduces to the *unconstrained* strictly convex quadratic

$$\min_{x_{\mathcal{F}}} \ \tfrac{1}{2}\, x_{\mathcal{F}}^\top A_{\mathcal{F}}\, x_{\mathcal{F}} - b_{\mathcal{F}}^\top x_{\mathcal{F}},$$

whose stationarity condition is the SPD linear system

$$A_{\mathcal{F}}\, x_{\mathcal{F}} = b_{\mathcal{F}}. \tag{4}$$

The active-set loop of Section 4 solves a sequence of such systems, one per outer step, and the inequality $x \geq 0$ is imposed by which indices $\mathcal{F}$ contains, never inside the linear solve. The inner solver is the proposed CG method.

**Matrix-free operator.** As a Krylov subspace method, CG requires only the mat-vec $v \mapsto A_{\mathcal{F}}\, v$ for $A_{\mathcal{F}}$, never the explicit matrix. On the free set $\mathcal{F}$ for $A = M^\top M$ this factors as $A_{\mathcal{F}}\, v = M_{\mathcal{F}}^\top (M_{\mathcal{F}}\, v)$ with $M_{\mathcal{F}} = M_{:,\mathcal{F}}$. These are two products with the $m \times |\mathcal{F}|$ submatrix at $O(m\,|\mathcal{F}|)$, so the Gram matrix is never formed and no $O(n^2)$ storage is used. This is equivalent to the standard CG-on-the-normal-equations arrangement [10, 25, 5] applied to a column subset that changes between outer steps.

The whole method requires only two operations of $A$, and this mat-vec is the first. The *free-block application* $v \mapsto A_{\mathcal{F}} v$ is all CG needs to solve (4). The *cross product* $x_{\mathcal{F}} \mapsto A_{\mathcal{B},\mathcal{F}}\, x_{\mathcal{F}}$ supplies the reduced gradient $s_i = (Ax)_i - b_i$ at the bound indices $i \in \mathcal{B}$, which the dual-feasibility test of Section 4 uses (since $x_{\mathcal{B}} = 0$). A full product $x \mapsto Ax$ is convenient for a residual check but not otherwise needed. Any backend realising these two operations runs the loop, its termination proof, and its convergence analysis unchanged. A dense $A$ slices them directly. The Gram operator $A = M^\top M$ evaluates them from $M$ without ever forming $A$, as above. A diagonal-plus-low-rank operator $A = \mathrm{diag}(d) + U \Delta U^\top$ inverts the free block by the Woodbury identity, a *direct* inner solve that bypasses CG. The equality-augmented problem (2) adds only the column selections $v \mapsto B_{\mathcal{F}} v$ and $w \mapsto B_{\mathcal{F}}^\top w$ of the Schur elimination below. When a variable is released in the primal step, the operator is rebuilt on the reduced free set and CG restarts on the smaller system. We now state the standard convergence result for CG.

**Proposition 3.1** (CG convergence [12, 29])**.** *Let $A_{\mathcal{F}}$ be SPD with spectral condition number $\kappa = \kappa(A_{\mathcal{F}})$, and let $x_k$ denote the $k$-th CG iterate for (4). Then*

$$\frac{\|e_k\|_{A_{\mathcal{F}}}}{\|e_0\|_{A_{\mathcal{F}}}} \;\leq\; 2\left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k,$$

*where $\|e\|_{A_{\mathcal{F}}} = (e^\top A_{\mathcal{F}}\, e)^{1/2}$ is the $A_{\mathcal{F}}$-energy norm of the error $e = x_{\mathcal{F}} - x_k$.*

The rate of convergence is thus $O(\sqrt{\kappa})$, so reaching a fixed relative energy-norm tolerance takes $O(\sqrt{\kappa}\,\log \varepsilon^{-1})$ iterations. The bound follows from a Chebyshev minimax argument: the $k$-th CG error is majorised by the best degree-$k$ polynomial on $[\lambda_{\min}, \lambda_{\max}]$ normalised at the origin, which is a scaled Chebyshev polynomial [29, Lecture 38]. Clustered spectra beat this bound. CG converges superlinearly once the extremal eigenvalues are resolved, which is exactly the behaviour the synthetic study of Section 6 measures. Because $A \succ 0$ forces every $A_{\mathcal{F}} \succ 0$, the bound applies at every outer step, and Section 5 shows how a regularising split lowers $\kappa$ and hence the count. This $\sqrt{\kappa}$ dependence is the whole reason to keep a Krylov inner solver rather than a plain gradient iteration: the projected-gradient reference method of Section 4 pays $O(\kappa)$ for the same per-step cost.

**Eliminating the equality multipliers.** For the equality-augmented problem (2), stationarity and feasibility on the free set are the $(|\mathcal{F}| + p) \times (|\mathcal{F}| + p)$ saddle-point system [2]

$$\begin{pmatrix} A_{\mathcal{F}} & -B_{\mathcal{F}}^\top \\ B_{\mathcal{F}} & 0 \end{pmatrix} \begin{pmatrix} x_{\mathcal{F}} \\ \lambda \end{pmatrix} = \begin{pmatrix} b_{\mathcal{F}} \\ c \end{pmatrix}, \qquad B_{\mathcal{F}} = B_{:,\mathcal{F}} \in \mathbb{R}^{p \times |\mathcal{F}|}. \tag{5}$$

Rather than solving the indefinite system directly, we eliminate $\lambda$ using the Schur complement. Stationarity gives $x_{\mathcal{F}} = A_{\mathcal{F}}^{-1}(b_{\mathcal{F}} + B_{\mathcal{F}}^{\top}\lambda) = v_0 + V_1\lambda$ in terms of the SPD solves

$$v_0 = A_{\mathcal{F}}^{-1} b_{\mathcal{F}} \in \mathbb{R}^{|\mathcal{F}|}, \qquad V_1 = A_{\mathcal{F}}^{-1} B_{\mathcal{F}}^{\top} \in \mathbb{R}^{|\mathcal{F}| \times p} \quad \text{(its } p \text{ columns, one CG solve each).}$$

Imposing $B_{\mathcal{F}} x_{\mathcal{F}} = c$ then fixes the multiplier through the $p \times p$ system

$$S\,\lambda = c - B_{\mathcal{F}} v_0, \qquad S = B_{\mathcal{F}} V_1 = B_{\mathcal{F}} A_{\mathcal{F}}^{-1} B_{\mathcal{F}}^{\top} \succ 0, \qquad x_{\mathcal{F}} = v_0 + V_1\lambda,$$

where $S$, the Schur complement, is SPD because $A_{\mathcal{F}} \succ 0$ and $B_{\mathcal{F}}$ has full row rank. The Lagrange multiplier is then $\lambda = S^{-1}(c - B_{\mathcal{F}} v_0)$, and the $p \times p$ solve can be done by a dense Cholesky decomposition at negligible cost.

The single-normalisation case $p = 1$, $B = \mathbf{1}^{\top}$, $c = \beta$ recovers the two solves $v_0, v_1 = A_{\mathcal{F}}^{-1}\mathbf{1}$ with the scalar $\lambda = (\beta - \mathbf{1}^{\top} v_0)/(\mathbf{1}^{\top} v_1)$. The pure-normalisation case $b = 0$, $\beta = 1$ collapses further to the single solve $x_{\mathcal{F}} = v_1/(\mathbf{1}^{\top} v_1)$. Thus each outer step costs $p + 1$ matrix-free CG solves. The multipliers are recovered analytically and never appear in the linear solve, so every inner system stays SPD and directly amenable to Proposition 3.1. The $p + 1$ right-hand sides share the operator $A_{\mathcal{F}}$, and can be solved by a block-CG or by independent CG runs, warm-started across outer steps.

# 4 Enforcing Non-Negativity

Section 3 reduces each candidate free set to an SPD solve, leaving $x \geq 0$ as the sole remaining constraint. Two strategies enforce it. The first is an active-set loop. It wraps the CG solver unchanged and certifies global optimality through the complementarity system (3). The second is projected gradient. It handles the constraint inside every step but forgoes the Krylov rate.

## 4.1 Active-set / block-principal-pivoting loop

The active-set method maintains a free set $\mathcal{F}$, solves (4) (or its equality-augmented form) on $\mathcal{F}$ by CG, and adjusts $\mathcal{F}$ toward feasibility. In the *primal step* any free variable that returned negative is pushed to the bound and dropped from $\mathcal{F}$. Once all free variables are non-negative, the *dual step* inspects the reduced gradient $s = Ax - b$ (minus $B^{\top}\lambda$ in the equality-augmented case, with $\lambda$ from Section 3) at every bound variable. A bound index $i$ with $s_i < 0$ would lower $f$ if released, so it is re-admitted to $\mathcal{F}$. The loop terminates when no variable violates either test, that is, when $x \geq 0$, $s \geq 0$, and complementary slackness hold to tolerance. These are exactly the KKT system (3), which for the strictly convex $f$ certifies the unique global minimiser.

Toggling an index between $\mathcal{F}$ and $\mathcal{B}$ is a *principal pivot* on $\mathrm{LCP}(A, -b)$, and dropping or adding several indices at once is a *block* principal pivot [18, 19]. Because $A \succ 0$ is a $P$-matrix, every principal submatrix is invertible and each pivot is well defined [22]. Block pivots are fast, but, like any all-at-once exchange, they can cycle: a batch drop can over-shoot the support and a later batch add restore it, revisiting a free set already seen. Algorithm 1 therefore runs the batch exchange as a *fast path* guarded by an anti-cycling counter. Let $N$ be the current number of primal and dual violators and $\bar{N}$ the fewest seen so far. A batch step is taken whenever it makes progress ($N < \bar{N}$) or a patience budget $p$ is unspent. Otherwise the algorithm falls back to a single *Bland-style* pivot on the lowest-indexed violator. This least-index single pivot is Murty's principal pivoting method, which cannot cycle [22]. It is what the finite-termination guarantee rests on; the batch path serves only to reach the optimum faster when it does not stall. This is the anti-cycling construction of Júdice and Pires for block principal pivoting [18].

**Algorithm 1** Active-set outer loop (wraps the CG inner solver $f$)

**Require:** Inner solver $f$ returning $(x_{\mathcal{F}}, k_{\text{step}})$ for free set $\mathcal{F}$; SPD $A$ (or its mat-vec) and $b$; tolerance $\varepsilon$; patience $p_{\max} \in \mathbb{N}$ *(default 3)*

**Ensure:** Minimiser $x \in \mathbb{R}^n$ of (1); outer steps $s$; total inner iterations $k$

1: $\mathcal{F} \leftarrow \{1, \dots, n\}$; $s \leftarrow 0$; $k \leftarrow 0$; $\bar{N} \leftarrow n+1$; $p \leftarrow p_{\max}$
2: **loop**
3: $(x_{\mathcal{F}}, k_{\text{step}}) \leftarrow f(\mathcal{F})$; $s \leftarrow s+1$; $k \leftarrow k + k_{\text{step}}$ *($k_{\text{step}}$: CG iterations; 1 for a direct inner solve)*
4: $x_i \leftarrow (x_{\mathcal{F}})_i$ for $i \in \mathcal{F}$; $x_i \leftarrow 0$ for $i \notin \mathcal{F}$
5: $s_i \leftarrow (Ax)_i - b_i$ *(reduced gradient; subtract $(B^\top \lambda)_i$ in the equality-augmented case)*
6: $\mathcal{D} \leftarrow \{i \in \mathcal{F} : x_i < -\varepsilon\}$; $\mathcal{V} \leftarrow \{i \notin \mathcal{F} : s_i < -\varepsilon\}$ *(primal and dual violators)*
7: $\mathcal{H} \leftarrow \mathcal{D} \cup \mathcal{V}$; $N \leftarrow |\mathcal{H}|$
8: **if** $N = 0$ **then**
9: **break** *(KKT (3) satisfied; globally optimal)*
10: **else if** $N < \bar{N}$ **or** $p > 0$ **then** *(fast path: progress, or patience remains)*
11: **if** $N < \bar{N}$ **then** $\bar{N} \leftarrow N$; $p \leftarrow p_{\max}$ **else** $p \leftarrow p - 1$
12: $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{D}) \cup \mathcal{V}$ *(batch exchange: drop all $\mathcal{D}$, add all $\mathcal{V}$)*
13: **else** *(anti-cycling fallback: $p$ exhausted)*
14: $i^\star \leftarrow \min\{i : i \in \mathcal{H}\}$ *(Bland least-index rule)*
15: **if** $i^\star \in \mathcal{D}$ **then** $\mathcal{F} \leftarrow \mathcal{F} \setminus \{i^\star\}$ **else** $\mathcal{F} \leftarrow \mathcal{F} \cup \{i^\star\}$ *(single principal pivot)*
16: **end if**
17: **end loop**
18: **return** $x$, $s$, $k$

**Theorem 4.1** (Finite convergence of Algorithm 1)**.** *Let $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ with $A \succ 0$. Then for any patience budget $p_{\max} \in \mathbb{N}$, Algorithm 1 terminates in finitely many outer iterations and returns the unique global minimiser of $\min_{x \geq 0} f(x)$. No non-degeneracy assumption is required. The same holds for the equality-augmented problem (2) with the reduced gradient of line 5 adjusted by $B^\top \lambda$, provided $B_{\mathcal{F}}$ retains full row rank on every free set the loop visits (automatic for $p = 1$; generic once $|\mathcal{F}| \geq p$).*

*Proof. Step 1 (each inner solve is well posed).* Because $A \succ 0$, every principal submatrix $A_{\mathcal{F}}$ is SPD, so $A$ is a $P$-matrix [22], and the system $A_{\mathcal{F}} x_{\mathcal{F}} = b_{\mathcal{F}}$ has a unique solution to which CG converges (Proposition 3.1). In the equality-augmented case the free-set solve is the saddle system (5); with $A_{\mathcal{F}} \succ 0$ and $B_{\mathcal{F}}$ of full row rank its Schur complement $S$ is SPD, so (5) has the unique solution recovered by the $p+1$ SPD solves of Section 3.

*Step 2 (termination implies global optimality).* Free-set stationarity gives $s_i = 0$ for $i \in \mathcal{F}$. The loop exits only when $N = 0$: (i) all free $x_i \geq 0$ and (ii) all bound $s_i \geq 0$. These are exactly $x \geq 0$, $s \geq 0$, $x_i s_i = 0$: the KKT system (3). Strict convexity of $f$ makes them sufficient for the unique global minimiser; it therefore suffices to show the loop cannot run forever.

*Step 3 (the fallback cannot cycle).* Call a maximal block of consecutive iterates taking the **else** branch (the single Bland pivot) a *fallback run*. Within a run $\bar{N}$ is constant, and each iterate performs one principal pivot on the lowest-indexed violator. This is precisely Murty's least-index principal pivoting method for the $P$-matrix LCP of Step 2, which generates a sequence of complementary bases (here, free sets) in which no basis recurs and which reaches the solution in finitely many pivots [22]. A fallback run is an initial segment of that sequence started from the free set left by the preceding step, so each run is finite, wherever it starts.

*Step 4 (only finitely many batch steps and runs).* The counter satisfies $0 \le N \le n$, and $\bar{N}$ is non-increasing, strictly decreasing exactly when the progress branch ($N < \bar{N}$) fires; hence that branch fires at most $n+1$ times. Each remaining fast-path step has $N \ge \bar{N}$ and decrements $p$, and $p$ resets only on a progress step, so at most $p_{\max}$ such steps occur between progress events. The number of fallback runs is likewise bounded: a run ends at termination ($N = 0$) or at a progress step, and there are at most $n+1$ progress steps. Thus the algorithm takes at most $(n+1) + (n+2)\,p_{\max}$ fast-path steps and at most $n+2$ fallback runs.

*Step 5 (finite termination).* By Step 3 each of the at most $n+2$ fallback runs is finite, and by Step 4 the fast-path steps are finite in number; their sum is finite, so the loop reaches $N = 0$ after finitely many outer iterations, and Step 2 certifies the returned $x$ as the unique global minimiser. A crude ceiling is the number of complementary bases, $2^n$. □

The $2^n$ ceiling establishes *finiteness* only; it is not an efficiency estimate. The guarantee is unconditional. The least-index fallback removes the non-degeneracy hypothesis that earlier active-set arguments need, and it is the fallback, not the batch exchange, that the proof rests on. In practice the fallback is rarely reached. A batch step fails to reduce $N$ only when a dropped index is immediately re-added (or vice versa), which forces $s_i = 0$ exactly for some toggled $i$, a codimension-one and hence non-generic condition [18]. Block principal pivoting therefore converges in a handful of outer steps on typical data, with the fast path alone certifying optimality. Section 6.2 exhibits the flip side: an anti-correlated design family that makes the event systematic, on which the unguarded batch path provably cycles and only the fallback terminates.

Theorem 4.1 assumes each inner call returns the exact minimiser of $f_{\mathcal{F}}$, whereas CG returns an iterate accurate only to its residual tolerance. The following lemma closes that gap. Once the residual is tight enough relative to the test threshold $\varepsilon$, every primal and dual sign decision of Algorithm 1 coincides with the decision the exact solve would make. The inexact loop then inherits the theorem wholesale.

**Lemma 4.1** (Inexact inner solves)**.** *For a free set $\mathcal{F}$ let $\hat{x}(\mathcal{F})$ denote the exact solution of (4) extended by zeros to $\mathbb{R}^n$, and $\hat{s}(\mathcal{F}) = A\,\hat{x}(\mathcal{F}) - b$ its reduced gradient. Define the* decision margin

$$\mu \;=\; \min_{\mathcal{F}\subseteq\{1,\dots,n\}} \min\Big(\big\{\,|\hat{x}_i(\mathcal{F})| : i \in \mathcal{F},\ \hat{x}_i(\mathcal{F}) \neq 0\,\big\} \cup \big\{\,|\hat{s}_i(\mathcal{F})| : i \notin \mathcal{F},\ \hat{s}_i(\mathcal{F}) \neq 0\,\big\}\Big),$$

*the smallest nonzero magnitude any primal or dual test can encounter; $\mu > 0$ as a minimum of finitely many positive numbers. Suppose the test tolerance satisfies $0 < \varepsilon < \mu$ and every inner CG call is stopped at a residual $\rho = \|b_{\mathcal{F}} - A_{\mathcal{F}}\tilde{x}_{\mathcal{F}}\|_2$ obeying*

$$\rho\,\frac{1+\lambda_{\max}(A)}{\lambda_{\min}(A)} \;<\; \min\big(\varepsilon,\ \mu-\varepsilon\big). \tag{6}$$

*Then at every outer step the violator sets $\mathcal{D}, \mathcal{V}$ of Algorithm 1 computed from the CG iterate coincide with those computed from the exact solve on the same free set. The inexact loop therefore traverses exactly the free-set sequence of the exact loop, Theorem 4.1 applies verbatim, and the returned iterate satisfies $\|\tilde{x} - x^\star\|_\infty \le \rho/\lambda_{\min}(A)$.*

*Proof.* We write $r = b_{\mathcal{F}} - A_{\mathcal{F}}\tilde{x}_{\mathcal{F}}$, so $\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}} = -A_{\mathcal{F}}^{-1}r$. From the Cauchy interlacing theorem we know that $\lambda_{\min}(A_{\mathcal{F}}) \ge \lambda_{\min}(A)$, hence

$$\delta_x \;:=\; \|\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}}\|_\infty \;\le\; \|A_{\mathcal{F}}^{-1}r\|_2 \;\le\; \rho/\lambda_{\min}(A).$$

For a bound index $i \notin \mathcal{F}$ the reduced gradients differ by $\tilde{s}_i - \hat{s}_i = A_{i,\mathcal{F}}(\tilde{x}_{\mathcal{F}} - \hat{x}_{\mathcal{F}})$, where $A_{i,\mathcal{F}}$ is the $i$th row of $A$ restricted to the free set. By Cauchy–Schwarz and $\|A_{i,\mathcal{F}}\|_2 \le \|A\|_2 = \lambda_{\max}(A)$,

$\delta_s := \max_i |\tilde{s}_i - \hat{s}_i| \le \lambda_{\max}(A)\,\rho/\lambda_{\min}(A)$. Both perturbations are bounded by $\delta := \rho(1 + \lambda_{\max}(A))/\lambda_{\min}(A) < \min(\varepsilon, \mu - \varepsilon)$ by (6).

We now fix $i \in \mathcal{F}$. If $\hat{x}_i \ge 0$ then $\hat{x}_i = 0$ or $\hat{x}_i \ge \mu$; either way $\tilde{x}_i \ge -\delta > -\varepsilon$, so the inexact test does not flag $i$, matching the exact test. If $\hat{x}_i < 0$ then $\hat{x}_i \le -\mu$ by the definition of $\mu$, so $\tilde{x}_i \le -\mu + \delta < -\varepsilon$: both tests flag $i$. The identical case split with $\hat{s}_i$ and $\delta_s$ handles the dual test. Hence $\tilde{\mathcal{D}} = \hat{\mathcal{D}}$ and $\tilde{\mathcal{V}} = \hat{\mathcal{V}}$ at this step; since both loops start from $\mathcal{F} = \{1, \dots, n\}$ and the counters $\bar{N}, p$ are functions of the violator sequence, induction gives the same free-set trajectory and the same stopping step. At termination the exact iterate is $x^\star$ (Theorem 4.1), so $\|\tilde{x} - x^\star\|_\infty = \delta_x \le \rho/\lambda_{\min}(A)$, the off-set entries being zero in both. □

**Remark 4.1** (Reading the margin condition)**.** *Three comments. (i) Components that are exactly zero at an exact solve are* not *excluded. The threshold $\varepsilon$ absorbs them on both sides of the comparison, so the lemma, like the theorem it protects, needs no non-degeneracy assumption. Moreover, since $\mu > \varepsilon$, the $\varepsilon$-thresholded tests of Algorithm 1 agree with the exact sign tests of the pivot argument in Theorem 4.1, so the algorithm as implemented is the algorithm as analysed. (ii) When CG is stopped at a* relative *residual $\varepsilon_{\mathrm{cg}}$, then $\rho \le \varepsilon_{\mathrm{cg}}\|b\|_2$ and condition (6) becomes a bound on $\varepsilon_{\mathrm{cg}}$ alone. (iii) The margin $\mu$ is not computable a priori, so the lemma is a license, not a recipe. It guarantees that* some *finite tolerance reproduces the exact trajectory, and in practice a residual tolerance a couple of orders of magnitude below $\varepsilon$ suffices (Section 6.2 verifies trajectory-for-trajectory agreement at $\varepsilon_{\mathrm{cg}} = 10^{-10}$, $\varepsilon = 10^{-8}$).*

### 4.2 Projected-gradient reference method

The second strategy enforces the constraint at every step. Projected gradient iterates

$$x_{k+1} = \Pi_C(x_k - \tau\,\nabla f(x_k))\,, \qquad \nabla f(x) = Ax - b,$$

where $C$ is the feasible set. For (1) it is the non-negative orthant, so $\Pi_C(y) = \max(y, 0)$ componentwise; for the single normalisation $\mathbf{1}^\top x = \beta$, $x \ge 0$ it is the simplex slice $\Delta_\beta$, projected onto in $O(n \log n)$ by a single sort-and-threshold pass [9, 6]. For a general equality system $Bx = c$, however, the projection onto $\mathcal{P} = \{x \ge 0 : Bx = c\}$ is itself a non-negative quadratic program with no closed form, so projected gradient loses its per-step simplicity. This is one reason the active-set loop is the method of choice once $p > 1$: its cost rises only by the $p$ extra right-hand sides of Section 3. The step size is $\tau = 1/L$ with $L = \lambda_{\max}(A)$, estimated once by power iteration on the same (matrix-free) operator. There is no outer loop and no linear solve: each step is one gradient evaluation (two products with $M$ when $A = M^\top M$) plus a projection.

**Proposition 4.1** (Projected-gradient convergence [23])**.** *Let $A \succ 0$, so $f$ is $\mu$-strongly convex and $L$-smooth with $\mu = \lambda_{\min}(A)$ and $L = \lambda_{\max}(A)$, and let $x^\star = \arg\min_{x \in C} f(x)$. Projected gradient with $\tau = 1/L$ satisfies*

$$\|x_k - x^\star\|^2 \;\le\; \Big(1 - \tfrac{1}{\kappa(A)}\Big)^k \|x_0 - x^\star\|^2, \qquad k \ge 0.$$

*Proof sketch.* $x^\star = \Pi_C(x^\star - \tau\nabla f(x^\star))$ is a fixed point; non-expansiveness of $\Pi_C$ and co-coercivity of the $L$-smooth gradient give $\|x_{k+1} - x^\star\|^2 \le \|x_k - x^\star\|^2 - \tau(2 - \tau L)\langle \nabla f(x_k) - \nabla f(x^\star),\, x_k - x^\star\rangle$. Setting $\tau = 1/L$ makes $2 - \tau L = 1$, and $\mu$-strong convexity bounds the inner product below by $\mu\|x_k - x^\star\|^2$, so $\|x_{k+1} - x^\star\|^2 \le (1 - \mu/L)\|x_k - x^\star\|^2$. □

The iteration count scales as $O(\kappa)$, against $O(\sqrt{\kappa})$ for the CG inner loop (Proposition 3.1). Both methods pay the same per-step cost (one operator application), so the $\sqrt{\kappa}$ gap in iteration counts is

a runtime gap on ill-conditioned problems. Nesterov acceleration would lift the projected-gradient rate to $O(\sqrt{\kappa})$, but CG attains that rate *optimally* within the Krylov subspace. Each CG iterate minimises $f$ over $\mathcal{K}_k(A_\mathcal{F}, b_\mathcal{F})$, a strictly richer search space than two-point momentum, and typically with a smaller constant. Plain projected gradient is therefore the honest first-order reference method, isolating the Krylov advantage; a FISTA variant [1] narrows the constant but not the asymptotic gap. The trade-off is otherwise in the other direction: projected gradient is loop-free and needs no complementarity certificate, so it is the simpler method when $\kappa$ is small or moderate accuracy suffices.

### 4.3 Warm-starting

For a parametric family of problems (e.g. a sequence $b(\theta_1), b(\theta_2), \ldots$ of right-hand sides, or a swept normalisation $\beta$), consecutive minimisers usually differ in only a few active constraints. Passing the previous problem's $(\mathcal{F}, x)$ pair as the initial state of the next solve reduces both the outer count (the free set needs fewer primal and dual adjustments) and the inner count (CG starts from a small initial residual, its guess being the previous solution restricted to the new free set). The support-stable case admits a clean statement.

**Proposition 4.2** (Warm starts across a support-stable step)**.** *Let $b' = b + \delta$ and suppose the free set $\mathcal{F}$ returned by Algorithm 1 for $b$ is also optimal for $b'$, i.e. the exact solve $\hat{x}_\mathcal{F} = A_\mathcal{F}^{-1} b'_\mathcal{F}$ is non-negative and its reduced gradient satisfies $s \geq 0$ on the bound set. Then Algorithm 1 started from $\mathcal{F}$ terminates in a* single *outer step, and its one CG solve, warm-started at the previous solution $x_\mathcal{F}$, reaches energy-norm accuracy $\tau$ in at most*

$$k \;\leq\; \left\lceil \frac{\sqrt{\kappa}+1}{2} \ln \frac{2\,\|\delta\|_2}{\tau\sqrt{\lambda_{\min}(A)}} \right\rceil \quad \textit{iterations:}$$

*the inner count scales with the logarithm of the* parameter step $\|\delta\|$*, not of the solution norm.*

*Proof.* The single solve on $\mathcal{F}$ returns $\hat{x}_\mathcal{F} \geq 0$ with $s \geq 0$ on the bound set, so both violator sets of line 5 are empty, the loop exits, and Step 2 of Theorem 4.1's proof certifies the global minimiser. For the iteration bound, the warm start's initial error is $e_0 = \hat{x}_\mathcal{F} - x_\mathcal{F} = A_\mathcal{F}^{-1}\delta_\mathcal{F}$, whose energy norm is $\|e_0\|_{A_\mathcal{F}} = (\delta_\mathcal{F}^\top A_\mathcal{F}^{-1}\delta_\mathcal{F})^{1/2} \leq \|\delta\|_2/\sqrt{\lambda_{\min}(A)}$ by Cauchy interlacing. Proposition 3.1 gives $\|e_k\|_{A_\mathcal{F}} \leq 2\rho^k \|e_0\|_{A_\mathcal{F}}$ with $\rho = (\sqrt{\kappa}-1)/(\sqrt{\kappa}+1)$, and $\ln(1/\rho) \geq 2/(\sqrt{\kappa}+1)$; solving $2\rho^k\|e_0\| \leq \tau$ for $k$ yields the bound. □

When the support does drift, no comparable a-priori bound is available; the loop is not monotone in the free set, as the adversarial family of Section 6.2 makes plain. The empirical picture is nonetheless simple: the warm-started outer count tracks the support drift $|\mathcal{F} \triangle \mathcal{F}'|$ and collapses to one whenever the drift is zero, exactly as the proposition prescribes (Section 6.2). A direct inner solver profits only from the warm free set, having no analogue of an initial guess. The matrix-free CG inner solver profits from both, which is where warm-started parametric sweeps obtain their largest speed-ups.

## 5 Regularisation and Preconditioning

Both the inner CG rate (Proposition 3.1) and the projected-gradient rate (Proposition 4.1) are governed by $\kappa = \kappa(A)$, so anything that compresses the spectrum of $A$ shortens the solve. Two mechanisms are available: a *regularising split*, which moves the operator toward a target the

application deems preferable, and *preconditioning*, which accelerates the solve of the operator as given. They coincide in a convenient way, developed in turn below.

### 5.1 Regularisation

We replace $A$ by the convex combination

$$A_\alpha = (1-\alpha)\,A + \alpha\,R^\top R, \qquad \alpha \in (0,1),\ R^\top R \succ 0, \tag{7}$$

a ridge/Tikhonov regularisation of $A$ toward the SPD target $R^\top R$. When $A = M^\top M$ this is again a Gram matrix, of the stacked operator

$$\tilde{M} = \begin{pmatrix} \sqrt{1-\alpha}\,M \\ \sqrt{\alpha}\,R \end{pmatrix}, \qquad \tilde{M}^\top \tilde{M} = A_\alpha,$$

so it is simply the Gram backend of $\tilde{M}$ (Section 3) and CG runs on $\tilde{M}_\mathcal{F}$ with no change. The augmented-matrix device is standard in least-squares regularisation [10, 20].

**Proposition 5.1** (Condition number under the regularising split)**.** *For $A_\alpha$ as in (7), Weyl's inequality [17] gives*

$$\kappa(A_\alpha) \;\le\; \frac{(1-\alpha)\lambda_{\max}(A) + \alpha\,\lambda_{\max}(R^\top R)}{\alpha\,\lambda_{\min}(R^\top R)}.$$

*The bound is strictly decreasing in $\alpha$; as $\alpha \to 1$ it approaches $\kappa(R^\top R)$, which equals 1 for $R = \sqrt{c}\,I$. For the scaled-identity target $R^\top R = \bar{\lambda} I$ it simplifies to $\kappa(A_\alpha) \le \frac{1-\alpha}{\alpha}\,\frac{\lambda_{\max}(A)}{\bar{\lambda}} + 1 = O\big((1-\alpha)/\alpha\big)$.*

The split works twofold. First, it lowers $\kappa$ and hence the iteration count of both solvers (Propositions 3.1 and 4.1). Second, because $A_\alpha \succ 0$ strictly whenever $\alpha > 0$, it restores the $P$-matrix property on which Theorem 4.1 depends, even when $A$ is only positive semidefinite (a rank-deficient least-squares operator with $m < n$). A strictly positive $\alpha$ is therefore the natural setting for the active-set loop: it simultaneously conditions the inner solve and guarantees that every principal submatrix is invertible.

### 5.2 Preconditioning

Regularisation lowers $\kappa$ by changing the operator. When no modelling change is admissible and $A$ must be solved as given, the same reduction in iteration count is available through preconditioning, which leaves the minimiser untouched. A symmetric positive definite preconditioner $P_\mathcal{F} \approx A_\mathcal{F}$ turns the inner solve (4) into preconditioned CG (PCG), equivalently CG on the symmetrically scaled operator $P_\mathcal{F}^{-1/2} A_\mathcal{F} P_\mathcal{F}^{-1/2}$, whose spectrum coincides with that of $P_\mathcal{F}^{-1} A_\mathcal{F}$. PCG realises this without ever forming $P_\mathcal{F}^{-1/2}$: each iteration adds a single application of $P_\mathcal{F}^{-1}$ to the one mat-vec with $A_\mathcal{F}$ [10, 2].

**Proposition 5.2** (Preconditioned inner rate)**.** *Let $P_\mathcal{F}$ be SPD and write $\kappa_P = \kappa(P_\mathcal{F}^{-1} A_\mathcal{F})$. The PCG iterates for (4) satisfy the energy-norm bound of Proposition 3.1 with $\kappa$ replaced by $\kappa_P$, so reaching a fixed relative tolerance costs $O(\sqrt{\kappa_P}\,\log \varepsilon^{-1})$ iterations. A preconditioner helps exactly when $\kappa_P < \kappa(A_\mathcal{F})$, and is ideal, $\kappa_P = 1$, when $P_\mathcal{F} = A_\mathcal{F}$.*

The bound is immediate: $P_\mathcal{F}^{-1/2} A_\mathcal{F} P_\mathcal{F}^{-1/2}$ is SPD and similar to $P_\mathcal{F}^{-1} A_\mathcal{F}$, so the two share the spectrum entering $\kappa_P$, and Proposition 3.1 applies to the scaled operator verbatim.

**Preconditioning on a changing free set.** Two constraints decide which preconditioners suit this method, both inherited from Section 3. First, the apply must be *matrix-free*. Forming $A_{\mathcal{F}}$ or a dense factor to build or invert $P_{\mathcal{F}}$ would forfeit the $O(m\,|\mathcal{F}|)$ operator and the absence of $O(n^2)$ storage that motivate the whole construction. Second, and particular to the active-set loop, the free set $\mathcal{F}$ changes between outer steps. Algorithm 1 rebuilds the operator on each reduced $\mathcal{F}$, so a preconditioner is cheap only if it *restricts* to the new free set at negligible cost. The standard choices sort by exactly this property.

The Jacobi preconditioner $P = \operatorname{diag}(A)$ restricts exactly, $(\operatorname{diag} A)_{\mathcal{F}} = \operatorname{diag}(A_{\mathcal{F}})$, so $P_{\mathcal{F}}$ is the free-set slice of a single vector computed once. For $A = M^\top M$ that vector is the squared column norms of $M$, available without forming $A$, and its apply is $O(|\mathcal{F}|)$. It carries no per-step setup and is valid on every free set the loop visits, the natural default when $A$ has a widely varying diagonal.

A factorised preconditioner (incomplete Cholesky, $P = LL^\top$) does *not* restrict. In general $(P_{\mathcal{F}})^{-1} \neq (P^{-1})_{\mathcal{F}}$, and the factor of the full $P$ is not a factor of $P_{\mathcal{F}}$, so honouring $A_{\mathcal{F}}$ would require refactorising on each free set, reintroducing the assembly the method avoids. Such preconditioners pay off here only once the free set stabilises, as it does near convergence and across the support-stable warm starts of Proposition 4.2, where one factor serves many outer steps.

Between these sits the low-rank-plus-diagonal preconditioner $P = \operatorname{diag}(d) + U\Delta U^\top$ built from a few ($r$) dominant eigenpairs of $A$ [27]. It restricts consistently in its *apply*, $P_{\mathcal{F}} = \operatorname{diag}(d)_{\mathcal{F}} + U_{\mathcal{F}}\Delta U_{\mathcal{F}}^\top$, and is inverted matrix-free in $O(|\mathcal{F}|\,r^2 + r^3)$ by the Woodbury identity, precisely the direct free-block solve for the diagonal-plus-low-rank case. It captures the leading spectral structure a diagonal misses. When $A$ is itself diagonal-plus-low-rank the preconditioner *is* $A$, giving $\kappa_P = 1$ and a direct inner solve that bypasses CG altogether.

The `nncg` method builds this low-rank-plus-diagonal preconditioner by randomized Nyström sketching [13], $A \approx U\Lambda U^\top + \mu(I - UU^\top)$ for an orthonormal $U \in \mathbb{R}^{n\times r}$, captured eigenvalues $\Lambda$, and a scalar tail shift $\mu$. This is the constant-diagonal case $d = \mu\mathbf{1}$, $\Delta = \Lambda - \mu I$ of the preconditioner above. It comes in two variants that trade off exactly where the free set changes. The first, `Nystrom`, sketches the *current* free block $A_{\mathcal{F}}$ on every outer step, using a handful of matrix-free products against $A_{\mathcal{F}}$ plus a QR and a small eigendecomposition. It is thus always built on the operator the loop is about to solve, at the price of repeating that work whenever $\mathcal{F}$ changes. The second, `GlobalNystrom`, sketches the *full* operator $A$ once. It exploits that restricting a rank-$r$ factorisation to a principal submatrix is exact, $(U\Lambda U^\top + \mu(I - UU^\top))_{\mathcal{F}} = U_{\mathcal{F}}\Lambda U_{\mathcal{F}}^\top + \mu(I_{\mathcal{F}} - U_{\mathcal{F}}U_{\mathcal{F}}^\top)$ with $U_{\mathcal{F}} = U[\mathcal{F}, :]$. Every later free set then reuses the one global sketch by masking rows rather than resketching. The masked $U_{\mathcal{F}}$ is no longer orthonormal, so its inverse needs the general Sherman–Morrison–Woodbury identity rather than `Nystrom`'s specialised identity-plus-projection form; it is built once per free set in $O(|\mathcal{F}|\,r + r^3)$ and applied in $O(|\mathcal{F}|\,r)$ thereafter. The trade mirrors the factorised-preconditioner case above. `GlobalNystrom` pays its larger, whole-operator sketch cost once, then amortises it across every outer step of a solve and across the support-stable warm-started re-solves of Proposition 4.2. That is exactly where a resketched-per-block `Nystrom` pays repeatedly for work `GlobalNystrom` does once.

**Stopping criterion.** PCG naturally monitors the preconditioned residual norm $(r^\top P_{\mathcal{F}}^{-1} r)^{1/2}$, whereas the transfer of finite termination to inexact solves (Lemma 4.1) is stated for the ordinary residual $\rho = \|b_{\mathcal{F}} - A_{\mathcal{F}}\tilde{x}_{\mathcal{F}}\|_2$ that enters condition (6). The two differ by a factor between $\lambda_{\min}(P_{\mathcal{F}})^{1/2}$ and $\lambda_{\max}(P_{\mathcal{F}})^{1/2}$. To keep the guarantee intact, the inner stop is evaluated on the unpreconditioned $\rho$ (or the preconditioned tolerance is tightened by $\kappa(P_{\mathcal{F}})^{1/2}$). Lemma 4.1 then applies word for word, because preconditioning changes only how fast $\rho$ falls, leaving $A$, the residual $\rho$ itself, and the decision margin $\mu$ untouched.

Two remarks connect the subsection to the rest of the paper. First, the regularising split (7) may itself be read as an implicit preconditioner. It changes the problem, but toward a target the application deems statistically or structurally preferable, so the conditioning gain comes at no modelling cost while additionally repairing rank deficiency. Second, preconditioning is a lever the Krylov inner solver has that the projected-gradient reference of Section 4.2 effectively lacks. A scaled gradient step minimises $f$ in the $P$-metric, but the orthant and simplex projections that make projected gradient cheap are closed-form only in the Euclidean norm, and become non-negative quadratic programs under a general $P$. Preconditioning therefore *widens* the $\sqrt{\kappa}$ advantage of the active-set/CG loop over projected gradient rather than closing it.

# 6 Complexity and Numerical Behaviour

## 6.1 Complexity

Table 1 collects the asymptotic cost of solving (1) by each method, for $A = M^\top M$ with $M \in \mathbb{R}^{m\times n}$, condition number $\kappa = \kappa(A_\alpha)$ after the regularising split, tolerance $\varepsilon$, free-set size $k = |\mathcal{F}| \le n$, and $s$ active-set outer steps. "Extra memory" is storage beyond the operator; the matrix-free rows never form $A$ and need only $O(n)$ working memory.

| Method | Per-step cost | Iterations | Extra memory | Forms $A$ |
|---|---|---|---|---|
| Interior point (dense QP) | $O(n^3)$ | $O(\log 1/\varepsilon)$ | $O(n^2)$ | Yes |
| Dense Cholesky + active set | $O(k^2 m)$ | $s$ | $O(k^2)$ | Yes |
| CG matrix-free + active set | $O(km)$ | $s \cdot O(\sqrt{\kappa})$ | $O(n)$ | No |
| Factor / Woodbury + active set | $O(kr^2 + r^3)$ | $s$ | $O(nr)$ | No |
| Projected gradient (matrix-free) | $O(nm)$ | $O(\kappa)$ | $O(n)$ | No |

Table 1: Asymptotic cost of solving the non-negative quadratic (1) with $A = M^\top M$, $M \in \mathbb{R}^{m\times n}$. Here $k = |\mathcal{F}| \le n$ is the free-set size, $s$ the number of active-set outer steps, $\kappa = \kappa(A_\alpha)$ the condition number after the split (7), and $r$ the factor rank of a diagonal-plus-low-rank operator. The last four rows share the active-set loop and differ only in the backend: matrix-free CG and the Woodbury factor solve avoid assembling $A$, whereas dense Cholesky forms and factorises $A_{\mathcal{F}}$. The CG "Iterations" column counts $s \cdot O(\sqrt{\kappa})$ inner iterations, dominated by the first solve at $k = n$; projected gradient pays $O(\kappa)$ at the same per-step cost.

Two comparisons are structural. Within the active-set family, matrix-free CG and dense Cholesky share the outer loop and differ only in whether $A_{\mathcal{F}}$ is assembled. The $O(km)$-versus-$O(k^2 m)$ gap means CG pulls ahead as the free set grows, while dense Cholesky can win when $k$ is small or $\kappa$ is large enough that CG needs many inner iterations (the crossover is near $\kappa \sim k^2$). Between the two matrix-free methods, CG's $O(\sqrt{\kappa})$ dominates projected gradient's $O(\kappa)$ whenever $\kappa$ is large. Only when the regularising split compresses $\kappa$ to $O(1)$ do the inner counts become comparable, and then projected gradient's loop-free simplicity is attractive.

The equality-augmented problem (2) multiplies the inner cost of an outer step by the $p + 1$ right-hand sides that share the operator $A_{\mathcal{F}}$ (Section 3), plus an $O(p^3)$ dense Schur solve that is negligible for the small $p$ of typical balance conditions. The asymptotics in $n$, $m$, and $\kappa$ are otherwise those of Table 1.

### 6.2 Behaviour on synthetic SPD problems

We test the propositions on a controlled family with a known optimum. Take $A = Q\Lambda Q^\top$ with $Q$ a random orthogonal matrix and a geometric spectrum $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$ on $[1, \kappa]$ of prescribed condition number $\kappa$ (equivalently $A = M^\top M$ for a random $M$ with the chosen singular values). Plant a non-negative solution $x^\star \geq 0$ of a chosen support by setting $b = Ax^\star - s^\star$ for a complementary $s^\star \geq 0$. Then $(x^\star, s^\star)$ is the exact solution of $\mathrm{LCP}(A, -b)$ in closed form. The runs below use $n = 200$ (support 50%) averaged over five seeds, with the smaller $n = 120$ where the projected-gradient $O(\kappa)$ cost forces a capped $\kappa$ range. All are reproduced by a self-contained NumPy script that implements Algorithm 1, the CG inner solve, and the reference method directly from their definitions. The matrix-free deblurring study is likewise NumPy-only. The external-solver benchmark below and the `shaw/phillips` ill-posed study are two further scripts whose only additional dependency is SciPy (and, for the benchmark, Clarabel). The algorithms are also released as the open-source package `nncg` (https://github.com/Jebel-Quant/nncg), whose test suite is this study. Planted recovery, the equality-augmented solve, trajectory agreement under inexact solves, the warm-start single-step property, the `shaw/phillips` regularisation and recovery runs, the matrix-free deblurring solve, and the adversarial family that forces the fallback all run as continuous-integration checks. The geometric spectrum is the honest choice: its clustering lets CG converge faster than the worst-case bound, so the measured growth is an *upper*-bounded confirmation of Proposition 3.1, not a fitted exponent.

| $\kappa$ | Outer steps $s$ | CG inner (total) | Fallback pivots |
|---|---|---|---|
| $10^1$ | 3.0 | 85 | 0.00 |
| $10^2$ | 3.2 | 212 | 0.00 |
| $10^3$ | 4.2 | 541 | 0.00 |
| $10^4$ | 5.0 | 1180 | 0.00 |
| $10^5$ | 5.2 | 2294 | 0.00 |
| $10^6$ | 6.2 | 4529 | 0.00 |

Table 2: Active-set solve of (1) on the synthetic family ($n = 200$, support 50%, mean over five seeds). Outer steps stay in single digits, and the total CG inner count grows sublinearly in $\kappa$ (slope 0.35 in log–log, $R^2 = 0.995$, against the $O(\sqrt{\kappa})$ worst case). The Bland fallback is never triggered here, so the fast block-pivot path certifies optimality alone. The solver recovers the planted $x^\star$ to $7e-10$.

Three predictions of the analysis are borne out.

*(i) Inner count is bounded by $\sqrt{\kappa}$; regularisation lowers it.* Figure 1 shows the total CG inner iterations rising with $\kappa$ but staying under the $O(\sqrt{\kappa})$ envelope of Proposition 3.1 (measured slope $0.35 < \frac{1}{2}$, the clustered spectrum buying CG its superlinear phase). The regularising split (7) with $R^\top R = I$ compresses $\kappa$ at rate $O((1-\alpha)/\alpha)$ (Proposition 5.1) and drives the count down monotonically as $\alpha$ increases (Figure 3). The effect is largest in the rank-deficient regime $m < n$, where only $\alpha > 0$ makes the system well posed (Table 3 below).

*(ii) Outer count is small and the fallback is dormant.* Across $\kappa \in [10, 10^6]$ the active-set loop terminates in at most 6 outer steps (Table 2), adjusting the free set by a few indices per step and reaching the correct support in a count that tracks its distance from the initial full set, not $n$. The patience budget is never exhausted: the Bland fallback of Theorem 4.1 guarantees termination but is not entered on generic data, exactly the codimension-one argument of Section 4.

*(iii) CG beats projected gradient by a widening $\sqrt{\kappa}$ factor.* Figure 2 plots both matrix-free methods at matched per-step cost. Both run faster than their worst-case bounds on the clustered spectrum, but projected gradient's growth exponent is about twice CG's, so their ratio grows

as $\kappa^{0.38}$ (essentially the predicted $\sqrt{\kappa}$) from parity at $\kappa = 10$ to 16× at $\kappa = 10^4$. This is the $O(\sqrt{\kappa})$-versus-$O(\kappa)$ separation of Propositions 3.1 and 4.1, widening with $\kappa$ and closing under heavy regularisation.

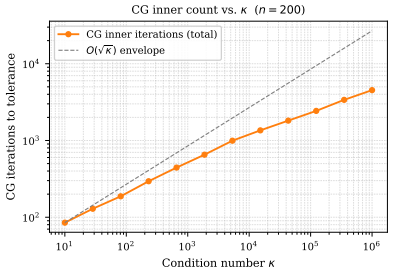


Figure 1: CG inner count vs. $\kappa$, under the $O(\sqrt{\kappa})$ envelope.

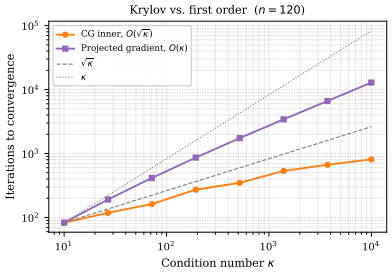


Figure 2: Krylov vs. first order: $O(\sqrt{\kappa})$ vs. $O(\kappa)$.

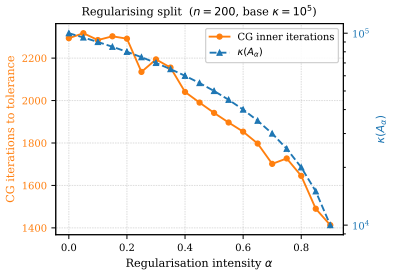


Figure 3: Regularising split: count falls as $\kappa(A_\alpha)$ shrinks.

*Exercising the fallback.* That the fallback lies dormant on generic data raises the question of whether it is needed at all. The following family answers it. Let the columns of $M$ arrive in near-anti-parallel pairs, $M = [\, M_0,\ -M_0 + \xi E \,]$, with small $\xi$ and a ridge to keep $A = M^\top M + \lambda I$ a $P$-matrix. Pushing a variable to the bound flips the sign of its partner's entry, so a batch drop systematically over-shoots and a later batch add restores it. This is the codimension-one near-tie event of Section 4 made systematic. On this family ($n = 20$, 60 seeds), *pure* block principal pivoting (Algorithm 1 with unbounded patience and exact inner solves) revisits a previously seen free set and cycles without terminating on 10 of 60 seeds. The guarded loop, exactly as stated (default patience, CG inner solves), converges on all 60. On 17 seeds the patience budget is exhausted and up to 22 least-index pivots are taken before the batch path resumes, each exit certified by the KKT system to working tolerance. The guarantee of Theorem 4.1 is therefore not decorative: on adversarial data the unguarded fast path genuinely cycles, and the fallback is what terminates it.

*Rank deficiency.* Table 3 probes the semidefinite regime the regularising split is claimed to repair: a Gram operator $A = M^\top M$ with $M \in \mathbb{R}^{100\times 200}$. Every free block with $|\mathcal{F}| > m = 100$ is then singular, including the initial one, since the loop starts from $\mathcal{F} = \{1, \ldots, n\}$. The failure at $\alpha = 0$ is instructive in its anatomy. When the planted support lies *below* the rank, the first inner solve can never meet its tolerance: the residual stalls at the component of $b_\mathcal{F}$ outside the range of $A_\mathcal{F}$, so the iteration cap is hit and the iterate carries no certificate. Yet the uncertified signs may still identify the support, after which the shrunken free blocks are regular. The loop then reaches the optimum on 4 of five seeds (at an order of magnitude more inner iterations) and fails to converge on the remainder. When the optimal support *exceeds* the rank, the final free block is itself singular, no certificate is ever available, and the loop converges on 0 of five seeds. Any $\alpha > 0$ restores the $P$-matrix premise of Theorem 4.1, and with it both reliability and speed: every run converges, recovering the planted optimum to $1e-09$. The partial successes at $\alpha = 0$ sharpen the point rather than soften it. What regularisation buys is not merely conditioning but the *guarantee*; without it, termination is luck.

*An official ill-posed problem.* The synthetic family is planted for control; to confirm the same behaviour on a recognised instance, we take the `shaw` problem from Hansen's Regularization Tools [14, 28], a severely ill-posed one-dimensional image-restoration model. Its exact solution, a sum of two Gaussians, is strictly positive and hence a genuine non-negative least-squares target. Its symmetric kernel $M$ is numerically rank-deficient, so the Gram operator $A = M^\top M$ ($n = 128$) is numerically singular, $\kappa(A) \approx 10^{20}$. Adding $10^{-3}$ relative noise to the data makes the unconstrained

| Support $k$ | $\alpha$ | Converged | CG inner | Capped solves | $\|x - x^\star\|_\infty$ |
|---|---|---|---|---|---|
| 50 | 0.00 | 4/5 | 2716 | 5 | $1 \cdot 10^{-9}$ |
| 50 | 0.05 | 5/5 | 190 | 0 | $6 \cdot 10^{-10}$ |
| 50 | 0.20 | 5/5 | 118 | 0 | $4 \cdot 10^{-10}$ |
| 150 | 0.00 | 0/5 | – | 5 | – |
| 150 | 0.05 | 5/5 | 351 | 0 | $1 \cdot 10^{-9}$ |
| 150 | 0.20 | 5/5 | 163 | 0 | $7 \cdot 10^{-10}$ |

Table 3: The rank-deficient regime ($n = 200$, $m = 100$, five seeds; CG capped at 2000 iterations per solve, 30 outer steps). "Capped solves" counts runs whose first inner solve hit the cap without meeting its tolerance. At $\alpha = 0$ the loop is unreliable below the rank and fails uniformly above it; any $\alpha > 0$ converges on every seed with an order of magnitude fewer inner iterations.

least-squares solution oscillate negative, so non-negativity binds as an active regulariser. Table 4 repeats the split sweep of Table 3 on this operator. At $\alpha = 0$ the loop cannot certify a solution: the singular free block leaves CG stalling on the inconsistent system, the outer budget is exhausted, and the KKT residual stays at $O(10)$. Any $\alpha > 0$ restores the $P$-matrix property, and the loop terminates in a handful of outer steps with a KKT certificate. At the representative $\alpha = 10^{-4}$ (which lowers $\kappa$ to $10^5$) it converges in 3 outer steps with 4 constraints active, agreeing with SciPy's Lawson–Hanson on the same regularised operator to $2e-04$. The speed advantage over Lawson–Hanson is the one the synthetic benchmark already records. The block loop reaches the 124-nonzero solution in 3 outer steps, whereas Lawson–Hanson admits a single index per pivot and so needs on the order of 124 of them. On the regularised operator this is a 50× wall-clock speedup at $n = 1024$, at an identical solution. The official problem tells the planted one's story: without the split, termination is luck; with it, the guarantee holds. A second Regularization Tools problem, `phillips` [14, 26] ($n = 512$), has a known non-negative solution that is exactly zero outside a central interval, so the active set is large and structured. The loop recovers the signal, certifies KKT optimality, and pins 273 variables at the bound against the 256 true zeros, agreeing with Lawson–Hanson to $1e-04$.

*Matrix-free at scale.* The problems so far are small enough to store $A$ densely. The method's reason for existing is the regime where that is impossible. A separable Gaussian blur $B = K \otimes K$ on an $N \times N$ image, applied as $X \mapsto KXK^\top$, gives a non-negative deconvolution [15] whose Gram operator $A = B^\top B$ has $n = N^2$ unknowns. The matrix-free loop (the operator solver of Section 3, reaching $A$ only through $v \mapsto B^\top(Bv)$) never forms it. At $N = 128$, so $n = 16384$, a dense $A$ would occupy 2.1 GB. With a ridge split for the $P$-matrix property, the loop instead runs in $O(n)$ working memory, terminating in 21 outer steps (3068 CG iterations total) with a KKT certificate and 13146 of 16384 pixels pinned at the non-negativity bound (Figure 4). Identical loop, identical proof; only the operator changes.

| $\alpha$ | $\kappa(A_\alpha)$ | certified | outer | CG inner | active | KKT resid. |
|---|---|---|---|---|---|---|
| 0 | $2 \cdot 10^{20}$ | no | 30 | 826 | 109 | $3 \cdot 10^{1}$ |
| $1 \cdot 10^{-6}$ | $8 \cdot 10^{7}$ | yes | 6 | 115 | 12 | $1 \cdot 10^{-9}$ |
| $1 \cdot 10^{-4}$ | $8 \cdot 10^{5}$ | yes | 3 | 48 | 4 | $2 \cdot 10^{-9}$ |
| $1 \cdot 10^{-2}$ | $8 \cdot 10^{3}$ | yes | 1 | 11 | 0 | $5 \cdot 10^{-10}$ |

Table 4: The `shaw` ill-posed problem from Hansen's Regularization Tools [14] ($n = 128$, $10^{-3}$ relative data noise; CG capped at 2000 iterations per solve, 30 outer steps). "Active" counts components pinned at the bound. At $\alpha = 0$ the singular Gram operator ($\kappa \approx 10^{20}$) leaves the loop uncertified; any $\alpha > 0$ terminates with a KKT certificate, matching Lawson–Hanson on the regularised operator.

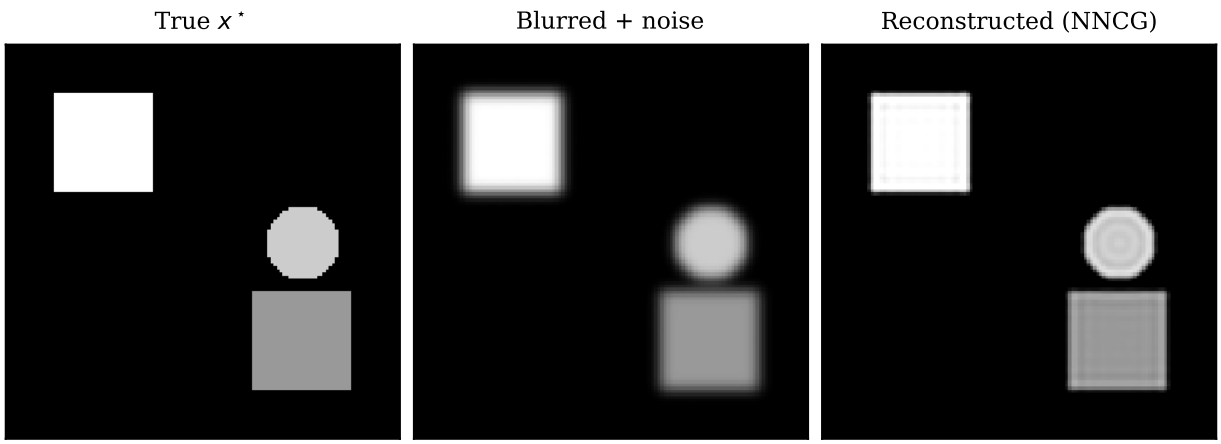


Figure 4: Matrix-free non-negative deblurring at $N = 128$ ($n = 16384$ unknowns): the true scene, its Gaussian blur with additive noise, and the reconstruction from the matrix-free active-set loop (Algorithm 1 with `solve_nnqp_op`), which reaches $A = B^\top B$ only through $v \mapsto B^\top(Bv)$ and never forms it, a dense $A$ being 2.1 GB. The black background is recovered by the 13146 active non-negativity constraints.

*Preconditioning.* Section 5's claim that a preconditioner slots into the loop unchanged is tested on operators with a well-conditioned core and a badly scaled diagonal, $A = D^{1/2}(Q\Lambda Q^\top)D^{1/2}$ with $\kappa(Q\Lambda Q^\top) = 10^2$ and the entries of $D$ spread over four decades. Jacobi-preconditioned CG inside Algorithm 1 runs at the core's conditioning regardless of the spread (326 inner iterations against 243 for the unscaled operator), while plain CG pays for the scaling in full (2983 iterations at $\kappa(A) \approx 9e+04$). That is a factor of 9 recovered by a preconditioner that costs one vector division per iteration.

*Inexact inner solves.* In line with Lemma 4.1, running the loop with its CG inner solver ($\varepsilon_{\text{cg}} = 10^{-10}$, $\varepsilon = 10^{-8}$) against a twin loop whose inner solver is an exact direct solve produces *identical* free-set trajectories on all 15 planted instances ($\kappa \in \{10^2, 10^4, 10^6\}$, five seeds each). The guarantee of Theorem 4.1 survives the inexact solves it is implemented with, not merely in principle but decision-for-decision.

*The general equality constraint.* The same loop with the Schur-complement elimination of Section 3 solves the equality-augmented problem (2) for a general $Bx = c$. On planted instances with $B \in \mathbb{R}^{p\times n}$ of full row rank, the solver recovers the optimum to $9e-09$ and meets the equality to machine precision for $p \in \{1, 3, 10\}$, with the inner cost scaling as the $p+1$ shared right-hand sides anticipated in Section 3. The case $p = 1$ reproduces the single-normalisation solve exactly.

*Warm-starting.* A sweep $b_k = b_0 + k\,\delta$ of 39 steps ($n = 200$, $\kappa = 10^4$, $\|\delta\| = 2\%$ of $\|b_0\|$) is solved cold and warm-started from the previous step's $(\mathcal{F}, x)$ pair (Figure 5). The mean outer count drops from 9.1 to 1.7 and the total inner count by a factor of 7.1. The mean support drift is 3.7 indices per step. The support-stable case of Proposition 4.2 is confirmed exactly: all 23 of 23 steps whose optimal support is unchanged are solved in a single warm-started outer step, and both starts return the same optimum on every step.

*External baselines.* The comparisons so far are internal: active-set variants against one another and against projected gradient. We close the section by placing Algorithm 1 among the external solvers a practitioner would actually reach for. This dense, explicitly assembled family is chosen to make that placement fair to the baselines, not to exhibit the matrix-free regime the method is built

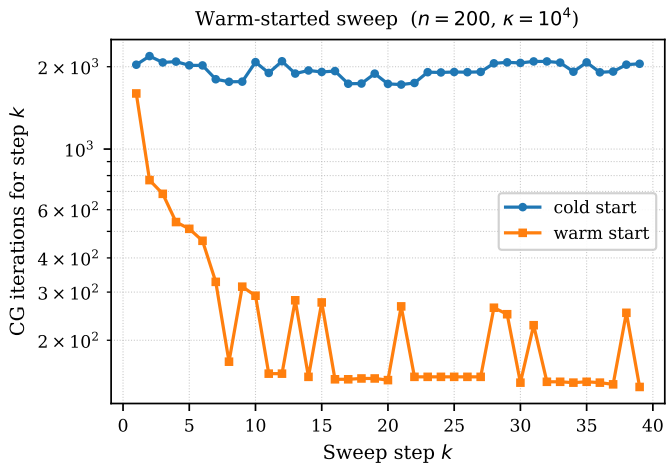


Figure 5: Per-step CG iterations along the parametric sweep, cold versus warm-started ($n = 200$, $\kappa = 10^4$). Warm starts pay only for the support drift and the $\log\|\delta\|$ of Proposition 4.2; the spikes are the steps where the optimal support moves.

for. On it the same planted family is solved by SciPy's Lawson–Hanson NNLS [20] (fed the Cholesky factor $M = L^\top$, factorisation included in the timing) and the interior-point solver Clarabel [11], against Algorithm 1 with a direct and with the matrix-free CG inner solve. Figure 6 reports wall-clock times against $n$ at $\kappa = 10^4$; Table 5 reports the $\kappa$-sensitivity at $n = 500$ and the accuracy of every solver against the planted optimum. Three observations follow. First, the active-set loop dominates both baselines on this family, at $n = 2000$ by a factor of 48 over Lawson–Hanson and 21 over Clarabel (direct inner solve). The reason is structural (Section 4): block pivoting reaches the optimal support in single-digit outer steps, where Lawson–Hanson moves one index per iteration and the interior point pays its Newton solves on the full $n \times n$ system. Second, the two inner solvers answer different questions, and this benchmark is the one that favours the direct block solve. It is faster in single digits at $n = 500$ (dense BLAS-3 factorisations beat memory-bound mat-vecs at these sizes) and is $\kappa$-insensitive, whereas the CG inner cost grows as $\sqrt{\kappa}$ (Table 5: a factor 20.5 from $\kappa = 10^2$ to $10^6$). This is by design, not a deficit. Where $A$ is small and dense enough to assemble and factorise, the direct solve should win, and the active-set loop (the paper's actual object) accommodates it unchanged: Theorem 4.1 governs the exact inner solve, and Lemma 4.1 carries the guarantee to the inexact CG one. The matrix-free CG inner solve earns its $\sqrt{\kappa}$ rate and $O(n)$ working memory only in the complementary regime, namely the Gram and structured operators of Section 3 that are never assembled, and warm-started parametric sweeps where each solve reuses the previous free set. Third, every solver recovers the planted optimum: Lawson–Hanson and the direct active set to $10^{-13}$, CG to its residual tolerance ($10^{-8}$), and Clarabel to its default interior-point tolerance ($10^{-4}$). The active-set methods certify KKT optimality exactly rather than to a duality-gap tolerance.

## 7 Conclusions

Conjugate gradients solves SPD systems; it does not, unaided, respect $x \geq 0$. This note supplies the missing layer. The bound-constrained quadratic $\min_{x \geq 0} \frac{1}{2} x^\top A x - b^\top x$, and its least-squares and equality-augmented ($Bx = c$) variants, reduces on any fixed free set to an unconstrained SPD solve

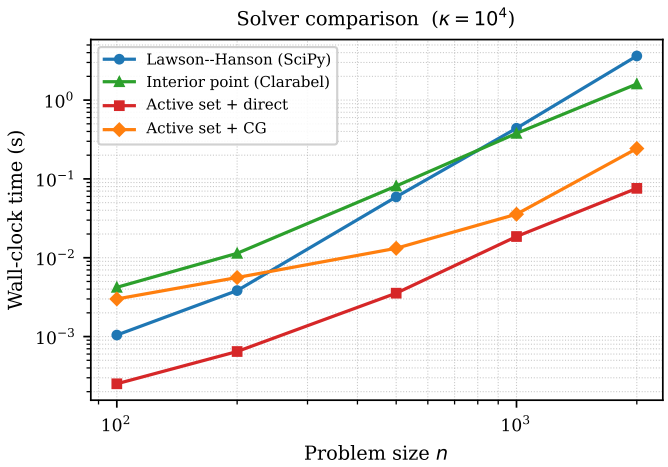


Figure 6: Wall-clock time against problem size $n$ on the planted family ($\kappa = 10^4$, support 50%, mean over seeds): SciPy's Lawson–Hanson NNLS and the Clarabel interior-point solver against Algorithm 1 with direct and matrix-free CG inner solves.

| | Time (s), $n = 500$ | | | |
|---|---|---|---|---|
| Method | $\kappa = 10^2$ | $\kappa = 10^4$ | $\kappa = 10^6$ | $\max\|x - x^\star\|_\infty$ |
| Lawson–Hanson (SciPy) | 0.051 | 0.060 | 0.069 | $5 \cdot 10^{-13}$ |
| Interior point (Clarabel) | 0.079 | 0.090 | 0.107 | $5 \cdot 10^{-4}$ |
| Active set + direct | 0.004 | 0.004 | 0.005 | $5 \cdot 10^{-13}$ |
| Active set + CG | 0.003 | 0.014 | 0.066 | $7 \cdot 10^{-8}$ |

Table 5: $\kappa$-sensitivity at $n = 500$ (mean over three seeds) and accuracy against the planted optimum. The direct methods are $\kappa$-insensitive at fixed $n$; only the CG inner cost grows, as Proposition 3.1 prescribes. Accuracy reflects each solver's own stopping criterion: the active-set exits are KKT certificates, the interior point stops at its duality-gap tolerance.

that CG performs matrix-free. Non-negativity is enforced around it by a primal-dual active-set loop whose asset toggles are the principal pivots of LCP($A$, $-b$). Because $A \succ 0$ is a $P$-matrix, guarding a fast block-pivot path with a least-index Bland fallback yields unconditional finite termination at the unique global minimiser (Theorem 4.1), with no non-degeneracy assumption. The inner solver retains CG's $O(\sqrt{\kappa})$ Krylov rate, a factor $\sqrt{\kappa}$ ahead of the loop-free projected-gradient reference method. A regularising split $A_\alpha = (1 - \alpha)A + \alpha R^\top R$ simultaneously compresses $\kappa$ and secures the $P$-matrix property, so it both accelerates the inner solve and licenses the termination proof.

The construction is deliberately application-agnostic: it takes an SPD operator (or a rectangular $M$ with $A = M^\top M$), a right-hand side, and an optional linear equality system $Bx = c$, and returns the non-negative minimiser. The guarantees and complexity recorded here hold for any SPD $A$. A reference implementation is available as the open-source package `nncg` (`https://github.com/Jebel-Quant/nncg`), whose test suite reproduces the synthetic study of Section 6.

## References

[1] A. Beck and M. Teboulle, *A fast iterative shrinkage-thresholding algorithm for linear inverse problems*, SIAM Journal on Imaging Sciences, 2 (2009), pp. 183–202.

[2] M. Benzi, G. H. Golub, and J. Liesen, *Numerical solution of saddle point problems*, Acta Numerica, 14 (2005), pp. 1–137.

[3] D. P. Bertsekas, *Projected Newton methods for optimization problems with simple constraints*, SIAM Journal on Control and Optimization, 20 (1982), pp. 221–246.

[4] R. Bro and S. De Jong, *A fast non-negativity-constrained least squares algorithm*, Journal of Chemometrics, 11 (1997), pp. 393–401.

[5] S.-C. T. Choi, *Iterative Methods for Singular Linear Equations and Least-Squares Problems*, phd thesis, Stanford University, 2006.

[6] L. Condat, *Fast projection onto the simplex and the $\ell_1$ ball*, Mathematical Programming, 158 (2016), pp. 575–585.

[7] Z. Dostál, *Box constrained quadratic programming with proportioning and projections*, SIAM Journal on Optimization, 7 (1997), pp. 871–887.

[8] Z. Dostál and J. Schöberl, *Minimizing quadratic functions subject to bound constraints with the rate of convergence and finite termination*, Computational Optimization and Applications, 30 (2005), pp. 23–43.

[9] J. Duchi, S. Shalev-Shwartz, Y. Singer, and T. Chandra, *Efficient projections onto the $\ell_1$-ball for learning in high dimensions*, in Proceedings of the 25th International Conference on Machine Learning (ICML), New York, NY, 2008, ACM, pp. 272–279.

[10] G. H. Golub and C. F. Van Loan, *Matrix Computations*, Johns Hopkins University Press, 4th ed., 2013.

[11] P. J. Goulart and Y. Chen, *Clarabel: An interior-point solver for conic optimisation problems with quadratic objectives*, INFORMS Journal on Computing, 36 (2024), pp. 481–499.

[12] A. Greenbaum, *Iterative Methods for Solving Linear Systems*, vol. 17 of Frontiers in Applied Mathematics, SIAM, Philadelphia, 1997.

[13] N. Halko, P.-G. Martinsson, and J. A. Tropp, *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions*, SIAM Review, 53 (2011), pp. 217–288.

[14] P. C. Hansen, *Regularization tools: A Matlab package for analysis and solution of discrete ill-posed problems*, Numerical Algorithms, 6 (1994), pp. 1–35.

[15] P. C. Hansen, J. G. Nagy, and D. P. O'Leary, *Deblurring Images: Matrices, Spectra, and Filtering*, SIAM, Philadelphia, PA, 2006.

[16] M. R. Hestenes and E. Stiefel, *Methods of conjugate gradients for solving linear systems*, Journal of Research of the National Bureau of Standards, 49 (1952), pp. 409–436.

[17] R. A. Horn and C. R. Johnson, *Matrix Analysis*, Cambridge University Press, 2nd ed., 2013.

[18] J. J. Júdice and F. M. Pires, *A block principal pivoting algorithm for large-scale strictly monotone linear complementarity problems*, Computers & Operations Research, 21 (1994), pp. 587–596.

[19] J. Kim and H. Park, *Fast nonnegative matrix factorization: An active-set-like method and comparisons*, SIAM Journal on Scientific Computing, 33 (2011), pp. 3261–3281.

[20] C. L. Lawson and R. J. Hanson, *Solving Least Squares Problems*, Prentice-Hall, Englewood Cliffs, NJ, 1974.

[21] J. J. Moré and G. Toraldo, *On the solution of large quadratic programming problems with bound constraints*, SIAM Journal on Optimization, 1 (1991), pp. 93–113.

[22] K. G. Murty, *Linear Complementarity, Linear and Nonlinear Programming*, vol. 3 of Sigma Series in Applied Mathematics, Heldermann Verlag, Berlin, 1988.

[23] Y. Nesterov, *Introductory Lectures on Convex Optimization: A Basic Course*, Kluwer Academic Publishers, Boston, MA, 2004.

[24] J. Nocedal and S. J. Wright, *Numerical Optimization*, Springer, New York, 2nd ed., 2006.

[25] C. C. Paige and M. A. Saunders, *Solution of sparse indefinite systems of linear equations*, SIAM Journal on Numerical Analysis, 12 (1975), pp. 617–629.

[26] D. L. Phillips, *A technique for the numerical solution of certain integral equations of the first kind*, Journal of the ACM, 9 (1962), pp. 84–97.

[27] Y. Saad, *Iterative methods for sparse linear systems*, SIAM, 2003.

[28] C. B. Shaw, Jr., *Improvement of the resolution of an instrument by numerical solution of an integral equation*, Journal of Mathematical Analysis and Applications, 37 (1972), pp. 83–112.

[29] L. N. Trefethen and D. Bau, III, *Numerical Linear Algebra*, SIAM, Philadelphia, PA, 1997.